Perceptions of Technical Debt and its Management Activities - A Survey of Software Practitioners

Danyllo Albuquerque
Federal University of Campina Grande (UFCG)
Campina Grande, Paraíba, Brazil
danyllo@copin.ufcg.edu.br

Everton Guimaraes
The Pennsylvania State University
Malvern, Pennsylvania, United States
ezt157@psu.edu

Graziela Tonin
Federal University of Fronteira Sul (UFFS)
Chapecó, Santa Catarina, Brazil
graziela.tonin@uffs.edu.br

Mirko Perkusich
Federal University of Campina Grande (UFCG)
Campina Grande, Paraiba, Brazil
mirko@virtus.ufcg.edu.br

Hyggo Almeida
Federal University of Campina Grande (UFCG)
Campina Grande, Paraiba, Brazil
hyggo@virtus.ufcg.edu.br

Angelo Perkusich
Federal University of Campina Grande (UFCG)
Campina Grande, Paraiba, Brazil
perkusich@virtus.ufcg.edu.br

ABSTRACT

Technical Debt (TD) is a metaphor reflecting technical compromises that can yield short-term benefits but might hurt the long-term health of a software system. Although several research efforts have been carried out, TD-related literature indicates that Technical Debt Management (TDM) is still incipient. Particularly in software organizations, there is still a lack of knowledge regarding how practitioners perceive TD and perform TDM in their projects. Our research focuses on characterizing TD and its management under the perspective of practitioners. For doing so, we conducted an online survey with 120 participants from 86 different organizations located in 5 different countries. Our results indicate that TD conception is widespread among more than 70% of respondents. Most of them (72%) recognized its importance and impact on software artifacts, being able to provide a valid example of three different TD Types (i.e., Design, Code, and Architectural). In addition, at least 65% of respondents consider TD identification, TD Repayment, and TD prevention as TDM activities in the spotlight. However, less than 15% adopt formal approaches to support these activities. This paper contributes to TD discussion and TDM activities by showing the practitioner's perspective. Finally, further research will support observing how effective and efficient TDM activities can be in different contexts.

CCS CONCEPTS

• **General and reference** → **Surveys and overviews**.

KEYWORDS

Technical Debt, Empirical Study, Survey, Technical Debt Management

SBES 2022, October 5–7, 2022, Virtual Event, Brazil

ACM ISBN 978-1-4503-9735-3/22/10…$15.00
https://doi.org/10.1145/3555228.3555237

ACM Reference Format:
Danyllo Albuquerque, Everton Guimaraes, Graziela Tonin, Mirko Perkusich, Hyggo Almeida, and Angelo Perkusich. 2022. Perceptions of Technical Debt and its Management Activities - A Survey of Software Practitioners. In *XXXVI Brazilian Symposium on Software Engineering (SBES 2022), October 5–7, 2022, Virtual Event, Brazil.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3555228.3555237

1 INTRODUCTION

Technical Debt (TD) was first introduced by Cunningham almost three decades ago [5]. TD is described as a collection of decisions that are expedient in the short term but set up a technical context that can make future changes more costly or even prohibitive [3]. Other researchers advocate that even though TD might also result from technical trade-offs that can yield short-term benefits (e.g., reduce time-to-market) but may harm the overall quality of a software system [10, 11, 18].

According to Martini [14], TD can be associated with different software artifacts (e.g., requirement, source code, and test artifacts) and span over different development phases. To deal with the TD occurrences, software organizations frequently carry out a set of activities that prevent potential TD from being incurred and assist in handling the accumulated TD to make it visible, controllable, and offset the software project's costs and value [11][1]. Technical Debt Management (TDM) includes critical activities associated with identifying, analyzing, monitoring, and measuring it [4].

TD is always present in real-world software development, and TDM is a recurrent process [3]. Many software organizations do not have established TDM practices, and practitioners alike are longing for methods and tools to help them strategically plan, track, and pay down TD items [11][1]. These organizations frequently need to consider the trade-offs between the overall software quality and the costs of the software development process in terms of required time and resources [15]. Despite the importance of TD, a few studies analyze how software organizations perceive and apply the TD metaphor in their working environment [2, 6]. Therefore, there is a latent gap regarding how software organizations perceive TD and how practitioners handle it in their software projects.

While TD concept and causes are an important topic [7], it is also worthwhile to understand how industrial practitioners take

TDM activities in software projects. Recently, some publications have some light and provided initial insights on this discussion [2, 6, 8, 13, 16, 19, 20]. However, the existing evidence is still limited since most of these studies focus only on a particular TD type to the detriment of the other TD types. In addition, they were interested in analyzing more in-depth a specific TDM activity. The discussion around TD perception and TDM activities performed by industrial practitioners deserves a more comprehensive investigation.

Our goal is to investigate overall aspects regarding TD and its management from practitioners in their software projects. Therefore, a survey was designed and conducted with practitioners engaged in multinational organizations. Based on the practitioners' perceptions, this study details the theoretical background on TD regarding its concepts, classification, TDM activities, and approaches for handling these activities. Finally, our study provides the following contributions: (i) a perception of the TD aspects and its management in multinational software organizations; (ii) a comparative analysis between previous related studies; (ii) we made available a survey package with empirically evaluated instruments to support and foster the study replication.

The remainder of this paper is structured as follows. Section 2 provides a background on TD concept, types and activities for its management. Section 3 present the main related studies. Section 4 presents the methodological configuration used to conduct this exploratory study. Section 5 presents and discusses the main findings. Finally, Section 6 presents the threats to validity and Section 7 presents the final remarks and possible future work.

2 BACKGROUND

This section presents the core concepts required to understand the present study. We discuss the fundamentals of TD, types, and activities required for its Management.

TD Fundamentals. Technical Debt (TD) is a metaphor reflecting technical compromises that can yield short-term benefits but may hurt the long-term health of a software system [3]. The term was introduced by Cunningham [5] when discussing with stakeholders the consequences of releasing poorly written code snippets to accelerate the development process. The TD metaphor is frequently used to communicate with non-technical stakeholders (e.g., corporate managers, clients, among others) about immature software artifacts and their impact. TD can be associated with the decision-making process about the shortcuts and workarounds taken in software development. However, these decisions can have a positive or negative influence on the quality of the software development artifacts. TD items can occur due to many factors, such as a lack of team members' knowledge in developing code compliant with the design or following a specific programming style. Therefore, software organizations must perceive and manage TD in their projects [20].

TD Types. TD can be associated with different software artifacts such as requirements document, architecture design, source code, and test artifacts. For instance, Li *et al.* [11] investigated how TD is spread throughout different development phases and collected a large number of TD types and related instances. Moreover, TD can be classified into ten coarse-grained types [11][4], and each of those is further broken down into several sub-types based on the causes. Next, we provide a simple definition of these TD types.

- *Requirements TD* refers to the distance between the optimal requirements specification and the actual system implementation, under domain assumptions and constraints;
- *Documentation TD* refers to incomplete or outdated documentation in any aspect of software development - i.e., outdated architecture documentation and absence of code comments;
- *Architectural TD* refers to architectural decisions that may compromise some internal quality aspects (e.g., maintainability);
- *Design TD* refers to technical shortcuts taken in detailed design (e.g., design or code smells).
- *Code TD* is caused by the poorly written code (e.g., code duplication and overly complex code) that violates best coding practices or coding rules;
- *Test TD* refers to shortcuts taken in testing. An example is the lack of software test documentation, or even when certain important types of tests are neglected (e.g., unit tests, integration tests, and acceptance tests);
- *Build TD* refers to flaws in a software system, its build system, or its build process, which makes the build overly complex and challenging;
- *Infrastructure TD* refers to a sub-optimal configuration of development-related processes, technologies, and tools, which negatively affects the team's ability to produce a quality product;
- *Versioning TD* refers to the problems in source code versioning, such as unnecessary code forks, or poor configuration management policy which might lead to conflicts between commits or pull requests when pushed to the source code repositories; and
- *Defect TD* refers to defects, bugs, or failures found in software systems during or after the development process.

These TD types will serve as a basis for our study to understand how practitioners define, know and perceive TD in their software projects. More detailed information about these TD Types can be found in the literature [4] [1].

TDM Activities. Technical Debt Management (TDM) includes activities that prevent potential TD from being incurred [11][4]. TDM also comprises those activities that deal with the accumulated TD to make it visible and controllable and to keep a balance between the cost and value of the software project. The activities involved in TDM can be summarized as:

- *Identification.* It detects TD caused by intentional or unintentional technical decisions in a software system through specific techniques, such as static code analysis;
- *Measurement.* It quantifies the benefit and cost of available TD in a software system through estimation techniques, or estimates the level of the overall TD in a system;
- *Prioritization.* It ranks the identified TD items according to specific predefined rules to support deciding which items should be repaid first and which TD items can be tolerated until later releases;
- *Prevention.* It implements action (e.g. use of guidelines, standards, or good practices) aims to prevent potential TD from being incurred on several software artifacts;

- *Monitoring*. It regards the monitoring of the unsolved TD items and their cost-benefit trade-offs throughout time.
- *Repayment*. It resolves or mitigates TD in a software system by techniques such as re-engineering and refactoring;
- *Representation/documentation*. It provides a way to represent and codify TD in a uniform manner addressing the concerns of particular stakeholders; and
- *Communication*. It makes identified TD visible to stakeholders so that it can be discussed and further managed.

Besides the TDM activities definitions listed above, Li et al. [11] provide a more in-depth definition about these ones. Similarly, the TDM categorization served as a basis for our survey to understand how software practitioners deal with TD in their projects.

3 RELATED WORK

This section discusses existing works reported in the literature and points out the current limitations addressed by our work. We describe each related study by following its chronological order as follows.

First, Ernst *et al.* [6] executed a survey with 1,837 participants in three organizations in the United States and Europe. They observed that the project context dramatically affects how the practitioners perceive TD. The authors observed that only 65% percent of the respondents adopted informal TDM practices in their projects. Next, Ampatzoglou *et al.* [2] conducted a study to understand how practitioners in organizations from the embedded systems domain perceive the TD. They performed an exploratory case study in seven organizations from four different countries. Among other research questions, the authors investigated what TD types frequently occur in embedded systems. Rocha and colleagues *et al.* [19] proposed a survey to understand how the TD is handled in practice. To this end, they identified the prominent facts that lead developers to create TD and which practices can prevent developers from doing so. Among the main reasons to incur TD, the participants answered management pressure, tight schedule, and developer inexperience.

Furthermore, Holvitie *et al.* [8] conducted a survey, including 184 practitioners from Finland, Brazil, and New Zealand. Approximately 20% of the participants had little to no knowledge of the TD definition. Thirty-five percent of the Brazilian participants were able to provide an example of a TD in their projects. According to the study, the leading causes of TD, selected by more than 50% of the participants, are associated with inadequate architecture, tests, documentation, software complexity, and violation of best practices or style guides. On the other hand, Silva *et al.* [20] conducted a survey with 62 practitioners, representing about 12 Brazilian software organizations and 30 software projects. The results indicate that TD is still unknown to a considerable fraction of the participants, and only a small group of organizations adopt TD management activities in their projects. The authors pointed out a set of technologies that can be used to support TDM activities and to make available a survey package to study TD and its management.

More recently, Mandić *et al.* [13] conducted a national-wide survey aimed to get empirical insight on the understanding and the use of the TD concept in the Serbian IT industry. The results indicate that the concept of TD is not widely accepted for use by the industry; only 35% of practitioners have practical experiences with projects that explicitly considered or managed TD. The most common TD types are code, test, and design debt, accounting for 61% of all reported cases. Similarly, Peres *et al.* [16] conducted a survey that focused on characterizing the practices related to TD payment. They analyzed responses from a survey of 432 practitioners from Colombia, Chile, Brazil, and the United States. They identified that refactoring (24.3%) was the primary practice related to TD payment, along with improving testing (6.2%) and improving design (5.8%).

We can conclude that several surveys with industrial practitioners were conducted to analyze TD under different aspects. Our work differs from the others because most studies mentioned above were interested in evaluating the TD concept, some particular TD type, or TDM activity. Our study aims to analyze more in-depth all TDM activities and evaluate the leading solutions employed for their support. Moreover, although the study by Silva *et al.* [20] analyzed all TDM activities, this work was limited to the viewpoint of Brazilian software organizations only. In this sense, the present study presents a more comprehensive analysis since it presents results regarding a more significant number of software organizations from different nationalities.

4 EXPERIMENTAL DESIGN

This section presents the research methodology adopted for the survey by following specific guidelines proposed by Pfleeger et al.[17]. Next, we describe the goals and research question of this study (Section 4.1), and provide a design of the questionnaire (Section 4.2). Then, we expose the procedures to execute a pilot test (Section 4.3). Finally, we present the employed data collection procedures (Section 4.4) and discuss the procedures applied to execute the survey (Section 4.5).

4.1 Goals and Research Questions

The goal of this study concerns to **analyze** the broader concept of Technical Debt (TD) and its management, **with the objective of** characterizing, regarding the level of knowledge, activities and technologies, from **the point of view** of software practitioners, **in the context** of software organizations. To address the study's general goal, we describe the Research Questions (RQ) as well as the motivation for each as follows.

- *RQ1- Is there a consensus on the overall aspects of TD among software practitioners?*. This question aims to collect and analyze TD concept, TD knowledge, and TD perceptions to determine if these aspects are homogeneous among practitioners in software organizations.
- *RQ2 - Which TDM activities are most relevant to software projects?*. The purpose of this question is to assess among practitioners what TDM activities are more relevant - or at least more considered - within their projects. Moreover, it is our goal to assess what activities are formally (or even informally) performed in real software projects.
- *RQ3 - What solutions are adopted for each TDM activity?*. This question aims to assess which solutions (e.g., techniques, practices, tools, or methods) are more commonly used to support practitioners when performing TDM activities.

Even though this survey was designed to identify the most common solutions used by professionals in software organizations as

means to support TDM activities, no judgment can be made about the efficiency and effectiveness of these solutions. It is important to note that does not intend to assess the benefits of applying such solutions in the context of software organizations.

4.2 Questionnaire Design

We designed the questionnaire following the guidelines presented by Linåker *et al.* [12]. Initially, we carried out an *ad-hoc* literature review to gather specific information about the perception and management of TD more broadly. Our analysis will be organized considering TD Types and TDM activities, similar to the classification frameworks proposed by Behutiye *et al.* [4] and Li *et al.* [11], respectively. These classifications are also aligned with the activities mentioned in other related studies [8, 13, 16, 19, 20]. Furthermore, our understanding is that these TD types and TDM activities are consistent since no disagreements were observed during the execution of the pilot tests.

For each TDM activity, we have identified a set of specific strategies and technologies used for their execution and a list of possible roles for each activity. Based on this information, a questionnaire was created, and we included specific questions for each activity. For example, in TD identification, a set of questions involving the classification of TD items was included - following the format proposed by Alves *et al.* [1].

Regarding the questionnaire structure, we decided to organize it into sixteen distinct sections (S) as described in Table **??**. The questionnaire is mainly composed of multiple-choice questions. A small number of open-ended questions were required to obtain more information from the participant. It also contains hybrid questions, meaning that these questions are multiple-choice with the ability to provide justification. These hybrid questions aimed to address issues related to tools and strategies associated with each of the TDM activities. This is due to the fact that the options provided may not cover all the possibilities of the participant's answers.

Agreement Section. Participants must accept the research conditions by declaring their agreement in the first section of the questionnaire (Section 0). Only participants that subscribed to these conditions could answer the next sections of the questionnaire.

Characterization sections. The three initial sections (Section 1-3) are related to the general characterization. First, the participants were asked about their age, role in the projects, academic formation, working experience in software projects, and other participants' profile questions. Then, the participants answered questions regarding its organization, such as its size, field, domain, and team size. Finally, the projects in which the participants work are also characterized through their duration, size, domain problem, and life-cycle model.

TD sections. Next, sections 4-6 focus on gathering the general participant understanding regarding the TD concept, knowledge, and perception. The first is associated with the participant's understanding of the TD concept. The second seeks to complement the participants' general understanding of TD and its characteristics. Finally, the third is associated with how TD items are manifested in the various software artifacts. It was not our purpose to inquire participants who are not familiar with the concept and meaning of TD, as they can provide wrong answers in the following sections. Instead, they were questioned if TD was perceived in their most recent project, i.e., if they could notice any issues associated with TD. An affirmative answer to this question allows the participants to answer two follow-up questions: if their organization adopts any TDM activity and if their manager (or themselves) adopts any TDM activity, regardless of their organization adopting any. Answering "yes" to any of these questions allows them to move forward to the next steps in the questionnaire.

TDM activities sections. The major part of the questionnaire (Section 7-15) asks for information regarding each TDM activity. They are only available to the participants if they select those activities in the previous section. At the beginning of each section, the TDM activity is described to improve the participants' knowledge and reduce the probability of misunderstanding the proposed question. All TDM activities subsections included a question to obtain which solutions were adopted to support that particular. TDM activity. It is essential to mention that at the end of each of the sections associated with TDM activities, an open question was made available for the participants to add (if necessary) more details about how they perform the TDM activity.

Completion Section. The last section (Section 16) was created to gather information regarding other TDM activities used in the participant's software organization, presenting similar questions to the previous sections. Additionally, if the participant wants to provide additional information to this research, he can use the open questions in this section.

4.3 Pilot Test

For the pilot study, our goal was to observe if the questionnaire was well understood by the participants. This survey was first designed in Portuguese since we initially targeted Brazilian organizations. However, we later decided to have the survey also applied in English to better understand how TDM is perceived in different countries. The survey translation required a validation phase that focused on reviewing whether the questions were consistent in both languages. This phase also executed an internal validation, an external validation, and a pilot study. This was done by the authors of this study with the help of external reviewers.

A pilot test was conducted using the same artifacts and procedures designed for the survey, including the questionnaire and method of execution, but with a small number of participants in the target audience [12]. Six practitioners with previous experience with TD - mainly in the industry - were invited to pilot tests. Three of them work on software projects in Brazilian software organizations. The other three also work on software projects in multinational software organizations. For the sake of privacy, we may not disclose the name of these companies.

Participants were invited to answer the questionnaire and return their comments regarding the response time, adequate understanding, integrity, lack of significant choices to answer, among other aspects. All pilot test participants responded to the survey within one week. The average response time was 25 minutes. The most relevant comments were associated with usability issues, clarity of questions, and some suggestions to improve some details and definitions throughout the questionnaire. These suggestions were later discussed and incorporated whenever applicable. Overall, we did

not observe negative comments or doubts about the answer choices or the descriptions of the questions, suggesting that the questionnaire was of sufficient quality and the necessary requirements to be used in the study.

4.4 Data Collection

We targeted only software software industry practitioners as respondents. The sample design adopted in the research is accidental and non-probabilistic (i.e., we cannot observe randomness in the selected population units). This decision may incur a threat to validity, which will be discussed later in Section 6.

A direct invitation to respond to the survey was sent through mailing lists that were forwarded to a number of software organizations and research groups in software development. This activity was supported by other researchers who made use of their primary network of contacts to maximize the number of respondents. Other invitations were sent through a professional social network *LinkedIn*. Finally, it is important to mention that we chose not to consider more than three responses from the same company to minimize bias in the results.

4.5 Survey Execution

The survey's goal is to achieve a deeper understanding of TD characterization and the main practices to address TDM activities. This survey can be classified as both exploratory and descriptive research because it was planned, structured, and focused on the discovery of insights, and the information collected can be statistically inferred over a population [21]. Due to the online nature of the survey, researchers did not intervene while participants answered the questionnaire. Despite the fact that this survey asks participants about a specific past experience related to a software project, it cannot be considered a cross-sectional study because respondents used different points in time [9].

The questionnaire was designed and configured to guarantee the anonymity of the participant. Google Forms was used as the platform to design and distribute it. Responses were automatically monitored and managed by Google, ensuring that no errors would be introduced when recording the responses. After the pilot study, the final survey was released in September 2020. The questionnaire provides a welcome message describing the survey structure and explains its importance to organizations. The participants were then asked to answer questions based on their current (or most recent) software project and organization. After the pilot study, the final survey was released on September 2020. The questionnaire innately provides a welcome message describing the survey structure and explains its importance to organizations. The participants were then asked to answer questions based on their current (or most recent) software project and organization. After answering questions related to overall TD aspects, each set of questions related to a specific TDM activity was conditionally presented to the participants. They answered only parts of the questionnaire in which they had some experience to minimize the problem of long survey questionnaires. Other conditional breakpoints were defined to terminate the survey if the participant is not familiar with the TD concept and if the organization or project does not apply any a specific TDM activity.

5 RESULTS AND DISCUSSION

The survey was carried out between October 2020 and September 2021. In total, 138 participants responded to the survey, with 120 complete responses from five different countries (Brazil, United States, Canada, Sweden, and India). We had 3 withdrawals and another 15 incomplete responses remaining, totaling 18 answers that were not included in the analysis. Due to space limitations, we provided a Supplementary Material containing the results not included herein [1]. Next, we will present the results associated with the: sample characterization (Section 5.1), overall TD aspects (Section 5.2), TDM activities (Section 5.3) and TDM solutions (Section 5.4)in the spotlight.

5.1 Sample Characterization

Based on the responses from the characterization sections (sections 1-3), we present the profile associated with the *participants*, the *organizations* and the most recent *projects* they have worked on.

Subjects Profile. More than 50% of respondents are aged between 30 and 39 years. Only four respondents reported having an incomplete undergraduate degree, while the remaining 116 respondents have at least an undergraduate degree. Thirty-three participants reported having a postgraduate degree at the *stricto sensu* level (i.e., master's or doctorate), while about 60% of the participants have an average work experience of more than 5 years in software projects. Less than 15% of respondents indicated they were "newbies" or "beginners" in terms of software development proficiency. Finally, about 70% of the respondents claimed to have the role of a developer, technical leader, or software architect.

Organizations Profile. Regarding the organizations that respondents work for, we noticed that most of these organizations (about 55%) are classified in one of the following sectors of activity: (i) Information Technology, (ii) Research, Development, and Innovation, or (iii) Finance. Regarding the organization's size, around 60% have up to 500 employees and only 25% of organizations have more than 2000 employees. Finally, most of these organizations (70%) have teams of up to 10 members. A minority of organizations have teams with more than 21 people (less than 15%).

Project Profile. Referring to the software project, most of them produce artifacts for the web (about 85%) and mobile devices (about 40%). Most of these projects (45%) have a problem domain associated with retail, finance, or business management systems. About 75% of the projects were more than one year old, and only 15% of these projects were more than 10 years old. Finally, 90% of the projects considered in this study use agile or hybrid approaches to development management.

It is worth saying that the sample considered meets the research target sample (See Section 4.4). Due to the anonymity of the questionnaire, it is not possible to specify the number of organizations (and their projects) represented in the survey. However, based on the information provided by the participants in this section, we can estimate the survey represents about 72 organizations and 97 projects included.

[1] https://doi.org/10.6084/m9.figshare.20346678.v1

5.2 Overall Aspects of TD (RQ1)

Based on the responses from the TD sections (sections 4-6), we will expose the general participant understanding related to TD and its overall aspect (i.e., concept, knowledge, and perception).

Regarding the TD concept, we presented to respondents a well known definition proposed by Avgeriou *et al.* [3]. Next, we asked the participants how close their understanding of technical debt was to the exposed concept. For almost 80% of the participants, the concept was "close" or "very close" to their understanding. These results demonstrate a disparity compared to the ones achieved by Holvitie *et al.* [8]. They pointed out that respondents were not "close" to the TD concept defined in that survey. This indicates that the TD concept is increasingly being spread among practitioners in software organizations.

Regarding knowledge about TD, from the 120 valid responses, only 12 respondents (10%) said they were not aware of the TD metaphor. From the 108 participants who claimed to know the concept of TD, the smallest part of them claimed to be aware of this concept from scientific materials such as articles or books (about 12%). For this sample, they had contact with the TD concept from previous experiences in software development projects (65%) and through blogs and discussion forums related to development (52%). The results obtained in this study are different from the results obtained in [8][20][13]. Finally, the results related to TD knowledge can be a good indicator that many industry practitioners have been aware about the occurrence and impact of TD items on their projects.

Regarding the TD perception, most respondents (72%) were able to provide a valid example of five different TD Types (i.e., Design, Code, Architectural, Requirements, and Test). This is a good indicator that even practitioners who had no prior knowledge about TD were able to understand the provided definition and associate it with their experience. This finding is very close to obtained by Mandic *et al* [13]. Moreover, most respondents (82%) associated the presence of *code smells* as the main manifestation of TD in their software projects. Furthermore, the imprecise requirements definition and low test coverage were other most recurrent forms of TD manifestation for 74% and 72% of respondents, respectively. These results are similar to the reported by Silva *et al.* [20] and distinct to those ones described by Mandić *et al.* [13].

Although most respondents stated that they perceive TD in their software projects, we noticed a lack of discipline in TDM activities. Only 11% of participants adopt a formal process for handling more than one TDM activity, possibly indicating the existence of a serious gap in the general perspective of product quality. This result is close to one obtained by Ernst *et al.* [6]. However, it is worth mentioning that we did not evaluate the adoption of other internal quality assurance methods to replace the low perception of the presence of TD. By grouping the five most important TD types from the respondents' point of view, we can identify the main symptoms associated with each of these types. Table 1 describes details of this analysis below:

We also noticed that managing TD items vary according to the organization's size. We could observe that a higher percentage of participants from organizations with a greater number of employees perform management in a more disciplined and formal way when compared to smaller organizations. This could indicate that larger organizations (generally being active for a longer period and having more solid processes to manage software projects) could have a broader perspective on the TD concept and its management. Similar findings were obtained in other studies related to the present research [6] [20].

Table 1: Symptoms Associated with TD Types.

TD Type	Symptoms Associated with TD
Code	Duplicate code (67%)
	Complex code (58%)
	Poor code quality (58%)
Design	Code Smells (79%)
	Complex classes and methods (64%)
	Frequent Changes (45%)
Test	Low test coverage (72%)
	Lack of specific test types (57%)
	Lack of tests in general (42%)
Architectural	Complex Architectural Dependencies (64%)
	Violation of good practices (57%)
	Low modularity (48%)
Requirements	Inaccurate definition of requirements (74%)
	Frequently changing requirements (65%)
	Customer does not know their needs (43%)

Finally, We can identify a similarity between the TD types most recurrently identified and managed by respondents and those they consider to be important. Figure 1 describes the 10 TD types according to their level of importance from the respondents' viewpoint.

5.3 TDM Activities (RQ2)

Based on the responses from the TDM sections (sections 7-15), we exposed the aspects involved in TDM activities. Figure 2 summarizes the responses associated with each TDM activity. The answers are described in percentage terms, indicating the number of participants who perform the activity formally (green), informal (yellow) and deliberately do not perform the activity (red).

As shown in Figure 2, all TDM activities were carried out by the participants. Identification and payment were the TDM activities most frequently performed and measurement and prioritization were the activities most neglected by respondents. No participant mentioned any other TDM activities as proposed in the questionnaire. These results were similar to Silva *et al.* [20] in terms of TDM activities in the spotlight. In addition, these results were similar to those described by Ernst *et al.* in terms of approaches to support TDM activities. In both studies, we can notice that most part of the respondents used informal approaches to perform TDM activities. In what follows, we will present the detailed results for each of the TDM activities:

TD Identification. About 70% of respondents, the identification of TD is carried out informally (i.e., without a minimally prescribed process for carrying out the activity). Another 17% indicated that this activity was not performed. For around 73% of respondents, the application of the TD identification strategy is optional. Regarding the tools and techniques for TD identification, manual code inspection (83%), static analysis (62%), and dependency analysis (42%) were the most frequently cited.

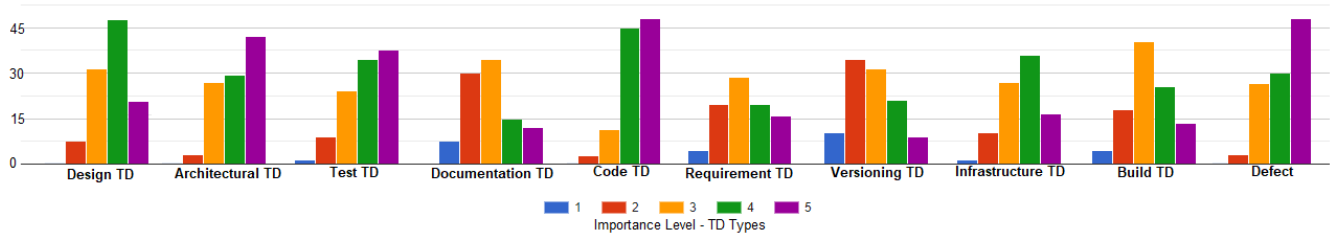

Figure 1: Importance Level - TD Types.

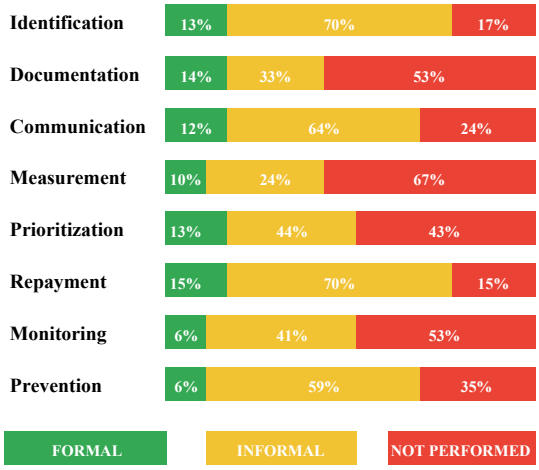

Figure 2: Overview of TDM activities.

Regarding the moment when the TD is identified, we noticed that for 62% of the respondents, the identification of TD items does not occur at "defined" moments. This sample only identifies TD items sporadically throughout the project or whenever they notice a problem. Additionally, respondents were allowed to explain how they believed it would be the "ideal" approach to identify the TD. For 86% of respondents, the TD identification should occur CONTINUOUSLY (i.e., as soon as the TD item is added, it should be identified and managed).

Although participants believe that TD items must be continuously identified and managed, for 66% of the sample of respondents, state-of-the-art tools and techniques do not adequately support this strategy. It is important to mention that a similar lack of tool support was presented by Ernst *et al.* [6]. Among the main benefits obtained with continuous identification are the non-accumulation of TD items (78%), greater ease in terms of time and effort for refactoring these items (69%), and greater transparency about the quality level of artifacts produced (67%).

Although the TD identification received a high level of interest from respondents, about 70% of respondents did not classify TD items after this activity. For 88% of respondents, code TD (i.e., code smells) are the most recurrent type of TD items identified and managed in their software projects. About 65% of respondents recognize that this type of TD occurs "frequently" or "very often".

For the open-ended question, most respondents stated that they identify TD items more frequently during the development of new features. This is typically accomplished at the end of development cycles (e.g., sprints) through static analysis tools, manual inspections, or code reviews.

TD Documentation. For about 53% of the respondents, the TD documentation is not carried out. For only 14% of respondents, this activity is carried out formally (i.e., with a minimally prescribed process for carrying out the activity). For these cases, around 52% carry out documentation with the support of a management tool. Lists or *backlogs* of items and *Issue Trackers* were the most used techniques for documenting TD in general for most respondents (62%). Moreover, many of the respondents stated that TD items are documented in tools external to the development environment, although most items are associated with the source code.

TD Communication. For about 64% of the respondents, this activity is carried out informally. About 24% do not perform TD communication. The most frequently used tools and techniques for communication were project meetings (45%) and lists of TD items exposed to the team (36%). In the open question, many of the participants reported communication problems between the parties involved regarding the presence of TD items.

TD measurement. For about 67% of respondents, this activity is not performed. For only 10% of the respondents, the TD measurement is carried out formally. Almost half of the respondents carry out the measurement from simple information (e.g., number of TD items). Regarding the information or criteria used to measure technical debt items, human resources (60%) and financial costs (29%) were the most recurrently mentioned by the respondents. The *Issue Tracker* tools (55%) and static code analysis (48%) got the most nominations. Finally, in the open question, some respondents reiterated that the measurement is not carried out objectively or rarely done - only when suitable to the project management team.

TD Prioritization. For about 43% of respondents, this activity is not performed. For only 13% of the respondents, the TD prioritization is carried out formally. For the sample that prioritizes TD items, product and business need (70%) and risk assessment (63%) were the most frequently used prioritization modes. Regarding the criteria, items that most impact the customer (82%) and items with a higher severity level according to the development team (76%) were the most used criteria. An interesting detail about prioritization is that part of the respondents stated in the open question that prioritized items were transformed into tasks to be included in the development cycle (e.g., *sprint*). Most of them reported that in

the intermediate stages of development, many of the activities are associated with solving TD items.

TD Repayment. About 70% of the respondents carry out this activity in an informal way. For only 15% of the respondents, the TD payment is carried out formally. A large part of the respondents (almost 70%) stated that the TD payment is planned according to the needs of the moment or the availability of time. For less than 10% of respondents, the TD payment is planned continuously, with specific periods of the development process dedicated to this activity. Code refactoring and rewriting are the most frequently used payment practices for 95% and 80% of survey participants, respectively. Maintainability, Reliability, and Evolving are the quality attributes most impacted by non-payment of technical debt items from the respondents' point of view. These results are in agreement with those obtained by Perez *et al.* [16]. Finally, in the open question, many of the payment actions for TD items were performed only when the development team noticed their effects.

TD Monitoring. For about 53% of respondents, this activity is not performed. For only 6% of the respondents, the monitoring of TD is carried out formally. Monitoring is performed only based on the number of technical debt items, and most respondents (about 70%) monitor TD items occasionally. Less than 20% of respondents stated that monitoring takes place on an ongoing basis and is based on available data on the occurrence of TD items. Manual inspection and Static code analysis were the most frequently used techniques for 74% and 65%, respectively. In the open question, respondents stated that monitoring is closely related to the prioritization of TD items. Many of them reported that TD items usually receive *tags* and these serve as a support to monitor and prioritize the transformation of these items into tasks.

TD Prevention. About 59% of respondents this activity is carried out informally. For 35% of respondents, TD monitoring is not performed. For most respondents (almost 60%), the standard practices for preventing TD items are only recommended. That is, they are not considered mandatory for all those involved. Code Reviews, Coding Patterns, and Automated Testing were the most commonly reported practices by respondents (88%, 80%, and 68%, respectively). In the open question, many of the respondents stated that a common strategy in the team to prevent the occurrence of TD items was to inspect/review the code before performing the *commit* to the code versioning system.

In general, our survey results show that there was no consensus on which TDM activities are most relevant to the software projects surveyed. Regarding the outcomes associated with these activities, our results are in agreement with the studies developed by Yli-Huumo *et al.*[23] and Silva *et al.* [20] which indicates that identification is most commonly adopted by practitioners, followed by payment and prioritization. Similarly, TDM activities rarely performed were measurement and monitoring, as also observed in our study. Another analysis that we were able to obtain with the research is directly related to the level of importance that each of the activities of the TDM process has according to the respondents' point of view. Figure 3 shows the results of the TDM activities considered "most important" by the participants of this research. We noticed that the identification and payment activities are, in fact, aligned with the results associated with the TDM activities. However, despite the number of participants indicating the importance of preventing TD (90%), less than 6% of participants perform this activity formally. This is a good indicator that the practices and perceptions of practitioners were different in the context of TDM activities.

5.4 Solutions for TDM Activities (RQ3)

Based on the responses from the TDM sections (sections 7-15), we exposed the main solutions (e.g., methods, tools, approaches, or techniques) to address TDM activities. Table 2 summarizes the most commonly used tod address TDM activities.

Table 2: Solutions to support TDM activities.

Activity	Adopted solutions
Identification	Manual Inspection (83%), Static Analysis (62%), Dependence Analysis (41%), CheckList (21%)and CheckStyle (19%)
Documentation	Lists or backlogs (62%), Issue Tracker (62%), management tool (24%), Spreadsheets (16%) and Wikis (8%)
Communication	Project Meetings (46%), Specific Meetings (37%), TD Lists (35%), Discussion Forums (25%) and Management Tools (18%)
Measurement	Issue Tracker (56%), Static analysis (48%), Manually (33%), Management tool (12%) and Wikis (7%).
Prioritization	Static Analysis (54%), Issue Tracker (48%), Manually (38%), Meetings (34%) and Management Tools (14%).
Repayment	Refactoring (95%), Code Rewriting (78%), Architectural Redesign (53%), Meetings (27%) and Artifact Change (15%).
Monitoring	Manually (74%), Static Analysis (66%), Issue Tracker (47%), Test Tool (26%) and Wiki (21%).
Prevention	Code Reviews (88%), Coding Standards (81%), Automated Tests (67%), Guidelines (62%) and Meetings (54%).

Based on the results presented in [23] [20], we note that there is a predominance of the use of static analysis tools for the identification of TD as well as three other TDM activities. It is well known that these tools perform analysis of source code structures to identify and warn of possible inconsistencies. This may indicate that this type of tool can be important for the correct identification and management of TD items, especially those associated with the source code. Another relevant aspect of using this type of solution is that it ideally should provide support for the continuous TD identification. Although around 85% of respondents believe that the TD identification should occur continuously (i.e., as early as if the TD item was added, it should be identified), what actually happens is that for more than 70% of the respondents, there are no pre-defined moments or they are only identified late when a problem is noticed. Similar findings were pointed out by Ernst et al. [6] in terms of lack of tooling support.

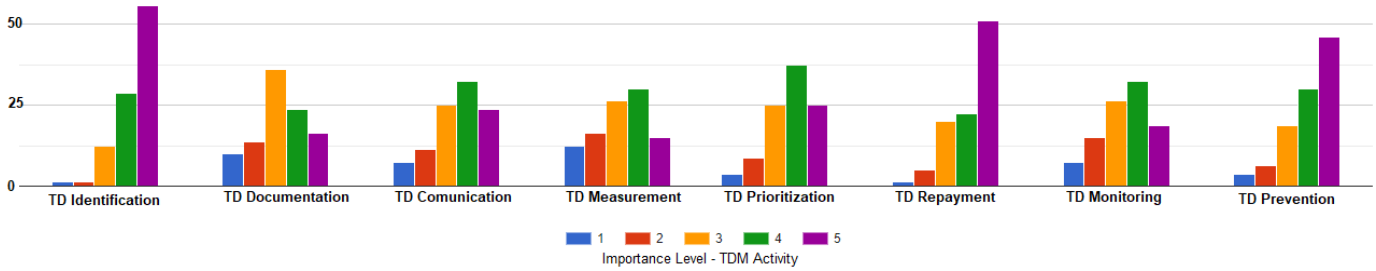

Figure 3: Importance Level - TDM activities.

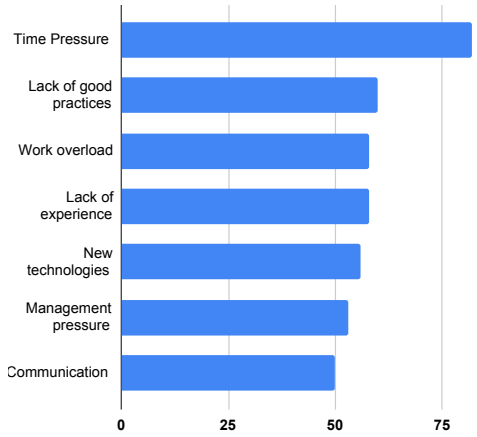

Figure 4: Causes of TD Occurrences.

Based on the participants' previous experiences, we asked which of the causes most contribute to the TD occurrence. The question presented a previous list of items. Respondents were also allowed to add any response outside the items on the list. The causes most frequently exposed by the participants are described in Figure 4. Time pressure was the most cited cause of TD occurrence by 82% of the respondents. Many of them mentioned a need for deliveries in increasingly shorter periods. This significantly impacted the quality of the artifacts produced. The lack of good programming practices (60%) was also often cited as a frequent cause of the incidence of TD items. Respondents associated the misuse of object-oriented principles and the lack of use of design patterns as factors that promoted the occurrence of TD items. Additionally, respondents mentioned that the absence of good practices degraded some software metrics (e.g., method size, class size) that implied in TD occurrences. It is worthy of noting these results are quite similar to those obtained by Rocha *et al.* [19] in terms of practices that should be adopted aims to avoid TD items.

6 VALIDITY THREATS

Next, we report this study's threats to validity by following the classification proposed by Wohlin *et al.* [22] with some actions taken to mitigate their effects.

Internal Validity. A potential threat is related to the participants' misunderstanding of the terms and concepts reported in the questionnaire. To minimize the effects, throughout the execution of the pilot tests, we refined the language to describe each question as well as their terms and concepts. Moreover, at the beginning of each section, the main terms necessary to understand the issues were exposed, including any additional concepts required.

Construct Validity. We identified a construct threat from the perspective of researchers and information collected from the technical literature. To reduce the impact of this threat, we conducted review cycles during research development with at least two researchers. In addition, pilot tests were performed, followed by a final review by all pilot test participants to ensure that the modifications aligned with their perspectives. We also noticed a potential threat in how the main research topic was communicated to participants through invitations. If the participants had no prior knowledge about the topic, this may have alienated them from the research, which could have influenced the results. We recognize that this effect of invitations on participants may have affected the study in some way.

External Validity. We observed a threat related to the representativeness and low adherence of the respondents invited to the survey. As part of the dissemination strategy involved presenting the form to research groups, researchers' networks, and in the professional network *LinkedIn*, some of our results may present some bias. Another threat may be associated with the size of the questionnaire. Overall, the questionnaire has 52 questions distributed in 16 sections. Although the questionnaire may be relatively large, not all respondents should answer all questions. The questionnaire was assembled with some conditional structures to allow respondents to navigate in a customized way along the questions.

Conclusion validity. The major threat arises from the coding process as a creative task during qualitative data analysis. The use of data analysis protocols with exact definitions of the analysis steps allowed the more rigorous review of the analysis steps by a second researcher. All inconsistencies were resolved via consensus.

Generalization Validity. Some threats limit the generalization of the research findings. We minimize this threat by distributing the survey geographically to get global coverage. Furthermore, we achieved a diversity of participants from different organization sizes, types of expertise, projects, and team sizes. We used convenience sampling, and as a result, the sample is slightly biased toward the developers' perspective (68% of participants are developers). Generally, it is hard to achieve generalizable results with a single

survey. Thus, this survey is designed to be replicated to achieve more creditable findings.

7 FINAL REMARKS

This study presented the results of a survey conducted with practitioners in software organizations from five different countries. The results obtained from 120 answers provide observations regarding how organizations perceive and manage TD in software projects. Regarding the TD concept, about 70% of the participants were "close" or "very close" to a well-known definition proposed by Avgeriou *et al.* [3]. Regarding TD knowledge, only 12 respondents (10%) said they were not aware of the TD metaphor. Finally, about TD perception, 72% of the respondents provided a valid example of three prominent TD Types (i.e., Design, Code, and Architectural). From respondents' viewpoint, identification, repayment, and Prevention were the most important TDM activities. These activities differ from ones that actually are performed formally by organizations. Regarding TD types, most respondents associated the presence of code smells (Design TD), the imprecise definition of requirements (Requirement TD), and low test coverage (Test TD) as the most recurrent TD types in their projects for 82%, 74%, and 72% of respondents, respectively.

In the context of TDM activities, we observed that only a few organizations (less than 15%) reported formal approaches to support these activities, indicating that TDM seems to be still incipient in organizations. Several factors such as developer skills, the organization's culture, and project domains can influence how TD can be perceived and managed by practitioners within their organizations. Based on this reasoning, since our sample differed from related studies, we noticed some discrepancies in the results compared to the existing literature. This strongly indicates a natural evolution in this research area, requiring additional empirical studies to complement the current knowledge about TD and its management. Please, note that it is not the scope of this paper to provide an in-depth comparison with the existing literature but yet demonstrate how the field is constantly growing.

Finally, we believe that the results of this study provide the following contributions: For practitioners, the results indicate that software practitioners and their organizations need to better understand better the TD concept. It is necessary to achieve better results in their projects since the TD perception and its management in this scenario are still incipient. For researchers, our results indicate that there is a need for more investigations aiming to disseminate the TD knowledge to practitioners on software organizations, as well as provide strategies and software technologies to support the TDM on these organizations.

ACKNOWLEDGMENTS

The authors thank all the professionals that took part in this survey. This research received support from the IFPB employee qualification incentive program (PIQIFPB) - Public Notice Nr 21/2021/PRPIPG.

REFERENCES

[1] Nicolli SR Alves, Thiago S Mendes, Manoel G de Mendonça, Rodrigo O Spínola, Forrest Shull, and Carolyn Seaman. 2016. Identification and management of technical debt: A systematic mapping study. *Information and Software Technology* 70 (2016), 100–121.

[2] Areti Ampatzoglou, Apostolos Ampatzoglou, Alexander Chatzigeorgiou, Paris Avgeriou, Pekka Abrahamsson, Antonio Martini, Uwe Zdun, and Kari Systa. 2016. The perception of technical debt in the embedded systems domain: an industrial case study. In *2016 IEEE 8th International Workshop on Managing Technical Debt (MTD)*. IEEE, 9–16.

[3] Paris Avgeriou, Philippe Kruchten, Ipek Ozkaya, and Carolyn Seaman. 2016. Managing technical debt in software engineering (dagstuhl seminar 16162). In *Dagstuhl Reports*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik.

[4] Woubshet Nema Behutiye, Pilar Rodríguez, Markku Oivo, and Ayşe Tosun. 2017. Analyzing the concept of technical debt in the context of agile software development: A systematic literature review. *Information and Software Technology* 82 (2017), 139–158.

[5] Ward Cunningham. 1992. The WyCash portfolio management system. *ACM SIGPLAN OOPS Messenger* 4, 2 (1992), 29–30.

[6] Neil A Ernst, Stephany Bellomo, Ipek Ozkaya, Robert L Nord, and Ian Gorton. 2015. Measure it? manage it? ignore it? software practitioners and technical debt. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*. 50–60.

[7] Yuepu Guo, Rodrigo Oliveira Spínola, and Carolyn Seaman. 2016. Exploring the costs of technical debt management–a case study. *Empirical Software Engineering* 21, 1 (2016), 159–182.

[8] Johannes Holvitie, Sherlock A Licorish, Rodrigo O Spínola, Sami Hyrynsalmi, Stephen G MacDonell, Thiago S Mendes, Jim Buchan, and Ville Leppänen. 2018. Technical debt and agile software development practices and processes: An industry practitioner survey. *Information and Software Technology* 96 (2018), 141–160.

[9] Barbara A Kitchenham and Shari Lawrence Pfleeger. 2002. Principles of survey research part 2: designing a survey. *ACM SIGSOFT Software Engineering Notes* 27, 1 (2002), 18–20.

[10] Philippe Kruchten, Robert L Nord, and Ipek Ozkaya. 2012. Technical debt: From metaphor to theory and practice. *Ieee software* 29, 6 (2012), 18–21.

[11] Zengyang Li, Paris Avgeriou, and Peng Liang. 2015. A systematic mapping study on technical debt and its management. *Journal of Systems and Software* 101 (2015), 193–220.

[12] Johan Linåker, Sardar Muhammad Sulaman, Rafael Maiani de Mello, and Martin Höst. 2015. Guidelines for conducting surveys in software engineering. (2015).

[13] Vladimir Mandić, Nebojša Taušan, and Robert Ramač. 2020. The prevalence of the technical debt concept in serbian it industry: Results of a national-wide survey. In *Proceedings of the 3rd International Conference on Technical Debt*. 77–86.

[14] Antonio Martini, Terese Besker, and Jan Bosch. 2018. Technical Debt tracking: Current state of practice: A survey and multiple case study in 15 large organizations. *Science of Computer Programming* 163 (2018), 42–61.

[15] Antonio Martini and Jan Bosch. 2015. Towards prioritizing Architecture Technical Debt: information needs of architects and product owners. In *2015 41St euromicro conference on software engineering and advanced applications*. IEEE, 422–429.

[16] Boris Pérez, Camilo Castellanos, Darío Correal, Nicolli Rios, Sávio Freire, Rodrigo Spínola, and Carolyn Seaman. 2020. What are the practices used by software practitioners on technical debt payment: results from an international family of surveys. In *Proceedings of the 3rd International Conference on Technical Debt*. 103–112.

[17] Shari Lawrence Pfleeger and Barbara A Kitchenham. 2001. Principles of survey research: part 1: turning lemons into lemonade. *ACM SIGSOFT Software Engineering Notes* 26, 6 (2001), 16–18.

[18] Nicolli Rios, Manoel G Mendonça, Carolyn Seaman, and Rodrigo O Spínola. 2019. Causes and effects of the presence of technical debt in agile software projects. (2019).

[19] Junior Cesar Rocha, Vanius Zapalowski, and Ingrid Nunes. 2017. Understanding technical debt at the code level from the perspective of software developers. In *Proceedings of the 31st Brazilian Symposium on Software Engineering*. 64–73.

[20] Victor Machado Silva, Helvio Jeronimo Junior, and Guilherme Horta Travassos. 2019. A taste of the software industry perception of technical debt and its management in Brazil. *Journal of Software Engineering Research and Development* 7 (2019), 1–1.

[21] FluidSurveys Team. 3. Types of Survey Research, When to Use Them, and How they Can Benefit Your Organization. *Retrieved December* 15 (3), 2017.

[22] Claes Wohlin, Per Runeson, Martin Höst, Magnus C Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in software engineering*. Springer Science & Business Media.

[23] Jesse Yli-Huumo, Andrey Maglyas, and Kari Smolander. 2016. How do software development teams manage technical debt?–An empirical study. *Journal of Systems and Software* 120 (2016), 195–218.